

# Reporting API - Example Routes

REST endpoints on `reporting.benyezhi.site`

Backed by MySQL ( `analytics` database).

- **Base URL:** `https://reporting.benyezhi.site`
- **Runtime:** Node.js / Express ( `127.0.0.1:3002` )
- **Content type:** `application/json`

## Routes

| Method | Route                                          | Description                                                                                                                                                            |
|--------|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET    | <code>/api/static</code>                       | Retrieve every static client record. Paginated via <code>?limit</code> & <code>?offset</code>                                                                          |
| GET    | <code>/api/static/{id   sessionId}</code>      | Retrieve one record by numeric <code>id</code> or session <code>id</code>                                                                                              |
| POST   | <code>/api/static</code>                       | Create a record (no <code>id</code> in the request).                                                                                                                   |
| DELETE | <code>/api/static/{id}</code>                  | Delete the record whose <code>id</code> matches                                                                                                                        |
| GET    | <code>/api/events</code>                       | Retrieve raw events. Filters: <code>session_id</code> , <code>type</code> , <code>start_time</code> , <code>end_time</code> , <code>limit</code> , <code>offset</code> |
| GET    | <code>/api/session/{sessionId}/timeline</code> | Device info plus the full time-ordered event list for one session                                                                                                      |

- GET without an `id` returns the collection; with an `id`, a single record.
- POST must not send an `id` since it is generated by the database. A `sessionId` field is required.
- Updates are handled by POST . There is no separate PUT route.
- DELETE requires an `id` in the path.

## Query parameters

| Param                    | Applies to                   | Default | Notes                         |
|--------------------------|------------------------------|---------|-------------------------------|
| limit                    | /api/static ,<br>/api/events | 50      | Capped at 100                 |
| offset                   | /api/static ,<br>/api/events | 0       | Row offset for pagination     |
| session_id               | /api/events                  | -       | Exact match                   |
| type                     | /api/events                  | -       | Exact match on event type     |
| start_time /<br>end_time | /api/events                  | -       | Bounds on<br>server_timestamp |

## Response envelope

| Field   | Notes                                                      |
|---------|------------------------------------------------------------|
| status  | "success" or "error"                                       |
| source  | "database" or "mock_fallback" - ** /api/static routes only |
| count   | Present on collection responses                            |
| data    | The record(s)                                              |
| message | Result text for DELETE and error responses                 |

## Example - POST /api/static

Request:

```
{ "sessionId": "postman-1", "userAgent": "Postman test", "language": "en-US",
  "cookiesAllowed": true, "screenDimensions": { "width": 1920, "height": 1080 } }
```

Response - 201 Created :

```
{ "status": "success", "source": "database",
  "data": { "id": 3, "sessionId": "postman-1", "language": "en-US" } }
```